

Towards Benchmarking Erasure Coding Schemes In Object Storage System

Rezuana Imtiaz Upoma^b, Md. Sadiqul Islam Sakif^{bc} and Jannatun Noor^{a,b}

^aDepartment of Computer Science and Engineering, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh

^bComputing for Sustainability and Social Good (C2SG) Research Group, School of Data and Sciences, BRAC University, Dhaka, Bangladesh

^cmPower-Social Enterprise, Dhaka, Bangladesh

ARTICLE INFO

Keywords:

Erasure Coding
OpenStack Swift
Reed-Solomon Codes
Cloud Storage System

ABSTRACT

Erasure Coding (EC) is a security measure that eliminates the need for complete data replication by allowing data to be reconstructed from parity pieces. This method offers increased data redundancy, efficiency, lowered storage costs, and enhanced fault tolerance making it a preferred alternative to replication in Object Storage Systems. The fundamental concept behind EC is to encrypt a specific amount of data in a manner that guarantees the transfer of all coded pieces without any loss. As data volumes continue to rapidly increase, the time efficiency of the EC method becomes crucial in ensuring optimal system performance. Various variables, including the algorithm employed, data size, number of storage nodes, hardware resources, and network conditions, can influence the speed of EC operations. Hence, we inspect the Erasure Coding algorithm known as Reed-Solomon Codes (RS). We explore the algorithm's encoding speed, taking into account factors such as data size and the generation of parity blocks. By evaluating and enhancing existing mechanisms, our research aims to address time efficiency and fault tolerance challenges in cloud-based object storage systems. To enhance data retrieval and storage processes, we comprehensively examine time efficiency mechanisms, such as data placement policies and scheduling algorithms. Additionally, we investigate the time taken for an open-source cloud storage system, OpenStack Swift, to store data by analyzing two datasets and determining the duration required to store these files. Furthermore, our study assesses fault tolerance mechanisms, including redundancy schemes, error correction codes, and distributed data placement strategies, to bolster system resilience. We propose different approaches to minimize access latency, improve overall time efficiency, and ensure data availability, even in the presence of failures.

1. Introduction

Distributed Storage Systems (DSS) play a pivotal role in addressing the formidable challenges posed by the unprecedented surge in data volume. They serve as indispensable solutions for scalable and dependable data storage [1, 2]. Incorporating Erasure Coding (EC) [3] within DSS is imperative, as it facilitates effective data distribution, minimizes storage overhead, ensures robust data durability, and fortifies fault tolerance. Thus, EC emerges as an intellectually sound choice for empowering large-scale storage systems. EC is a reliable and efficient storage solution for the modern data-driven world. It addresses the limitations of traditional methods by offering time efficiency, user-friendliness, and fault tolerance. EC disperses data across multiple devices and generates redundant parts, ensuring data integrity even in the face of device failures. This reduces the risk of data loss.

Additionally, EC's lower storage overhead improves system performance and user experience by providing redundancy with a smaller footprint. Its transparent data recovery eliminates the need for significant system modifications or

user training, making it user-friendly. Examining the performance of Erasure Coding schemes is crucial as it enables companies to make informed decisions and contribute to data storage advancements. Erasure coding holds great promise in eliminating the need for redundant storage and addressing challenges related to fault tolerance and time efficiency. The realm of EC research encompasses more than mere fundamental comprehension and a general survey of Erasure Coding principles. In fact, these scholarly inquiries delve into benchmarking erasure codes with an emphasis on data security and an array of benchmarking factors to assess their efficacy.

Moreover, researchers have devised solution-based approaches aimed at augmenting the performance and effectiveness of Erasure Coding techniques. To evaluate the Erasure coding, Atish et al. [3] evaluate popular distributed storage systems using diverse criteria and explore the integration of Erasure Codes. Through a comprehensive survey, it highlights design principles, data models, failure handling, consistency, and security. Kevin et al. [4] investigate the potential of Erasure Codes to reduce power consumption in storage systems. The authors introduce power-aware coding, a technique that utilizes the structure of erasure codes to achieve energy savings.

Besides, Balaji et al. [5] examine efficient repair techniques in distributed storage systems, including regenerating codes, locally recoverable codes, and novel repair methods for Reed-Solomon codes. Cong et al. [6] present a flexible distributed storage integrity auditing mechanism for secure

 rezuana.imtiaz.upoma@bracu.ac.bd (R.I. Upoma);

md.sadiqul.islam.sakif@bracu.ac.bd (Md.S.I. Sakif);

jannatun.noor@bracu.ac.bd (J. Noor)

ORCID(s): 0000-0003-3113-8603 (R.I. Upoma); 0000-0001-9669-151X (J. Noor)

and dependable cloud storage, utilizing homomorphic tokens and distributed erasure-coded data. To enhance the performance of EC, Yan et al. [7] present Coop-U, a cooperative update scheme for efficiently updating erasure-coded data in distributed storage systems.

To tackle the privacy issue in distributed networked storage systems, Hsiao et al. [8] propose a secure decentralized erasure code that combines threshold public key encryption and a variant of decentralized erasure code. The resulting storage system is decentralized, robust, private, and cost-effective, ensuring data privacy even when all storage servers are compromised. Xiaoqiang et al. [9] propose Group-U, a grouped and pipelined update scheme for erasure-coded storage systems. It achieves significant reductions in update overhead and time compared to existing approaches, improving efficiency by dynamically adjusting group size and optimizing data transmission and computation. Additionally, there are some papers that work on Benchmarking Erasure Coding in terms of cost analysis [10], failure repair [11], storage efficiency [[12], [10], [13]], data availability and durability [14] and fault tolerance [[14], [13]]. However, none of the studies focus on finding time efficiency in terms of I/O operations of EC schemes.

Benchmarking EC is vital as it allows for the evaluation of performance, optimization, scalability assessment, comparative analysis, and standardization. It provides insights into the efficiency and speed of Erasure Coding operations, helps identify areas for improvement, and enables the selection of the most suitable solution. By benchmarking, organizations can assess system performance under different workloads, compare different implementations, and contribute to the development of standardized benchmarks.

EC is the preferred choice among developers in various scenarios. In cloud storage systems, it is favored for its ability to ensure data durability and cost-effectiveness. In distributed file systems, EC is implemented to enhance fault tolerance and scalability. Data archiving and backup solutions benefit from EC as it enables efficient storage. In large-scale storage systems, it optimizes resource utilization. When it comes to data center storage infrastructure, EC is relied upon to maintain high availability. Lastly, in the realm of IoT applications, Erasure Coding is utilized to guarantee reliable data storage. Traditional cloud systems are facing challenges in handling the rapid growth of data in today's distributed application environment. While replication is commonly used for redundancy, Erasure Coding emerges as a valuable alternative [3].

EC's technique of dispersing the data over numerous devices and producing redundant parts ensures that the data's integrity will be preserved even in the event that one or more of those devices fails. Because of its capacity for fault tolerance, the possibility of data being lost is reduced. In addition, EC's lower storage overhead enhances both the performance of the system as well as the user experience. This is due to the fact that it offers redundancy while maintaining a smaller footprint. Because of its transparent data recovery, it is easy to use since it removes the need for significant system

modifications or user training. This makes it user-friendly. Examining the performance of Erasure Coding schemes is essential because it enables companies to make informed decisions and facilitates the progress of data storage technology.

Erasure Coding has emerged as an enticing solution for the challenges of fault tolerance and time efficiency. Our work aims to prioritize the identification of EC methods that can optimize both the time efficiency and fault tolerance of OpenStack Swift cloud storage systems. Among XOR-based Erasure Codes [15], Fountain Codes [16], Luby Transform (LT) Codes [17] and Reed-Solomon Codes (RS) EC techniques, we choose RS codes for benchmarking because of the limitations on fault tolerance, inefficiency with large data sets, and error correction capability. Our work focuses on experimenting with diverse datasets to benchmark Erasure Coding (EC) schemes in Object Storage System (name OpenStack Swift). This involves selecting an appropriate EC technique, creating an experimental testbed, and carefully choosing datasets for analysis and comparison. We experiment with multiple implementations to analyze the data and achieve desired results.

Based on our study, we make the following contributions to this paper:

- At first, we investigate the time efficiency of Erasure Coding (EC) using Reed Solomon Codes in the OpenStack Swift Object Storage System.
- Then, we measure upload, download, and deletion times for specific Erasure Coding schemes (RS (5+3), RS (10+4), and RS (7+5)) [18], and provide unique insights and a comparison of their time efficiency.
- Besides, we explore the fault tolerance aspect of Erasure Coding using the SimEDC simulator and compare fault tolerance results with various schemes to improve reliability.
- Moreover, we create our own Erasure Coding dataset (MCSD-100) and use another benchmark dataset (COCO-17) for comprehensive analysis and validation.
- Finally, we present a comparative analysis with other relevant papers. Our experimentation validates the uniqueness of our research and identifies opportunities for enhancing time efficiency and fault tolerance in Erasure Coding.

Overall, our contributions include in-depth time efficiency analysis, fault tolerance exploration, dataset utilization, comparison with existing studies, and identification of opportunities for improvement in Erasure Coding for cloud-based object storage systems.

The organization of this paper is further segmented into different sections. Section 2 explores related literature and research papers in the field. In Section 3, the background of Erasure Coding is discussed, with a focus on RS for OpenStack Swift and the SimEDC simulator used in the testbed.

Section 4 details the methodology employed, including the creation of test bed environments and the selection process for Erasure Coding. It also covers the datasets used in the testbed. Section 5 provides a description of the datasets utilized. Section 6 presents the experimental evaluation and analysis of results. comparison with related works. Section 7 presents the comparative analysis with related work and limitations.

2. Related Work

In the field of Cloud Computing, there are challenges in terms of developing an improved security system in a distributed storage system. Many challenges with new technologies are being overcome one day after another. But behind all those successes, many researchers have studied and developed different theories and methodologies. Storing big data in OpenStack Swift [19] with the help of Erasure Coding (EC) has also been successful on many occasions. Those achievements were pursued as previous authors have mentioned and came up with different ideas and techniques. Such reviews are given below:

2.1. Storage System

Wang et al. [27] inspect the distributed storage systems, point out the problems with replication, and suggest Erasure Coding (EC) as a way to deal with security issues, data placement, transfer costs, and the costs of keeping copied data up-to-date. Opara-Martins et al. [32] use cloud tools to handle big amounts of social media data. They suggest using a multi-node OpenStack cloud system to give Hadoop as a service, making it easy to use within the OpenStack cloud setting. Zhang et al. [33] come up with a way to update data quickly in big-data distributed storage systems. They use a protocol that uses locking and caching to make sure that changes to data and code blocks are always the same. Heo et al. [34] focus on SSD-based cloud storage systems and present a method called “parity database reduction” for OpenStack cloud storage platforms. This method lowers needless parity data and extends the life of SSDs in all-flash arrays.

Besides, Song et al. [25] suggest FACHS (File Access Characteristics based Hybrid Storage), an adaptive hybrid storage system that optimizes storage redundancy, parallel read/write performance, and storage capacity using a combination of Reed-Solomon (RS) Code, multi-copy, and LRC codes based on file access patterns. This improves efficiency, speed, and storage utilization. Rivera et al. [35] propose Fulcrum coding, a hybrid coding method that uses random linear network coding and systematic coding to improve multicast services in different networks by improving FEC efficiency, reliability, transfer speed, and decoding rates. Shin et al. [36] suggest a cache-based matrix method to improve the speed of global file systems that store large amounts of data. This can be done by saving and reusing matrices in cache memory, which reduces costs and improves efficiency. Using the Hadoop Distributed File System

(HDFS) [37] and RS code in experiments has shown shorter times to write, read, and restore.

2.2. Replication and Deduplication

Nandyal et al. [30] compare replication and Erasure Coding (EC) algorithms in distributed file systems with an emphasis on data availability and stability. They address the performance issues associated with EC by proposing a buffering and combining mechanism to manage encoded I/O requests and by supplying recovery strategies for decoding problems in EC-based distributed file systems. Lombardo et al. [20] propose a “parity data deduplication” technique designed expressly for flash-based RAID [38] storage, especially all-flash arrays in the context of OpenStack cloud block storage, which reduces the need for parity data write operations. Guo et al. [21] present a deduplication scheme for secure key sharing and ownership management in cloud storage that makes use of elliptic curve cryptography and broadcast encryption for enhanced security and efficiency. In addition, Calheiros et al. [39] present a proactive replication method that substantially reduces cloud data center storage waste, network traffic, and energy consumption.

2.3. Erasure Coding

Chouhan et al. [40] and Abebe et al. [41] examine erasure encoding techniques, specifically Reed-Solomon (RS) coding, to determine optimal parameters for dependable and cost-effective storage systems, taking into account storage overhead, data accessibility, and retrieval. Cheng et al. [42] present StreamLEC, a fault-tolerant stream machine learning system employing Erasure Coding (EC) for low-redundancy proactive fault tolerance and enhanced performance in failure recovery and latency. Nachiappan et al. [43] propose a recovery strategy employing Erasure Coding in data centers to improve dependability with minimal overhead by optimizing data block selection for proactive replication based on failure predictions, resulting in substantial reductions in storage costs, network usage, and energy consumption. Qin et al. [44] propose enhancing the efficacy of the Ceph [45] system’s EC mode by selecting storage nodes based on resource demand and diversity, thereby enhancing read/write performance.

Moreover, Iliadis et al. [46] examine the effect of secret sector defects on erasure-coded replication in large-scale storage systems and emphasize the significance of accurately positioning data to improve reliability. Arslan et al. [22] discuss reliability prediction for erasure-coded storage systems in warm/cold storage, investigating stochastic models, a new Markov model, multi-dimensional Markov modeling, and simulation modeling to address system maintenance and enhance forecasting. Chiniah et al. [3] address the difficulties of managing growing data volumes in cloud systems and propose EC as an alternative to replication in distributed storage, evaluating various storage methods and highlighting the benefits of integrating EC into distributed storage systems. Pu et al. [47] propose EMS, a novel UHD video streaming system that combines multi-source streaming and erasure-coded storage to improve the quality of UHD video in 5G

Reference	Purpose of the study	Technology used	Brief of Findings
Lombardo et al., 2023 [20]	RAID storage frequent parity data updates	All-flash arrays, redundant parity data	Reducing parity data write operations compared to standard data duplication
Guo et al., 2023 [21]	Cloud encryption and title management	Elliptic curve cryptography (ECC)	Direct uploader-cloud service provider deduplication improves system security and eliminates third-party intervention
Arslan et al., 2023 [22]	Erasure-coded warm /cold storage systems' availability and endurance	Stochastic models, Markov models	Detachable drive medium Markov models
Levitin et al., 2023 [23]	Assessing production dual storage system mission success probability (MSP)	Probabilistic model for MSP	Optimizing production storage systems
Geetha et al., 2023 [24]	Create a reliable cloud storage solution	DSS encryption	Time-based data forwarding re-encryption
Song et al., 2023 [25]	Help distributed storage systems to retrieve batch -stripe data after data loss	Reed-Solomon code	BPR outperforms rPDL (Repair-by-Download).
Yin et al., 2023 [26]	Optimize networked storage systems for 'cold' data storage utilizing erasure codes	Erasure code strategies, replication strategies, consistent hashing	System performance
Tang et al., 2023 [21]	Overcome distributed storage system network transmission bandwidth constraints that prevent erasure code use	Three-hierarchical redundancy structure	TLRC's data recovery procedure uses less time and data than other erasure codes
Wang et al., 2022 [27]	Compare DSS	Erasure Coding	DLOP-partitioning based storage scheme
Tanwar et al., 2022 [28]	Data partitioning and deployment	Distributed Linear Order Partition (DLOP)	DLOP and partitioning based storage scheme
Wang et al., 2022 [29]	Manage rapid data changes in heterogeneous wide stripe erasure coded storage system	G-Update outperforms other schemes	Time-based data forwarding re-encryption
Nandya et al., 2021 [30]	DFS data availability and stability	Replication and Erasure Coding	EC saving space over replication
Siagian et al., 2021 [31]	Video surveillance database system architecture network performance improvement	OpenStack	OpenStack in IaaS
Our Work	Time efficiency and Fault Tolerance measurement	Reed Solomon, OpenStack Swift, Erasure Coding	Less time efficient, More fault tolerant

Table 1
Comparison between research studies

networks, addressing storage and bandwidth constraints and employing federated learning and a Multi-Armed Bandit approach to improve service quality and user experience. Table 1 summarises the research works and compares them.

2.4. OpenStack Swift

Emu et al. [48] present an enhanced strategy for shared-mode resource allocation for cloud-based load testing utilizing OpenStack Swift as the cloud server and Apache JMeter as the load testing instrument. They conduct experiments using a variety of scenarios and metrics to assess system

behavior and determine load capacity scaling capacity. Kengond et al. [49] advocate the use of cloud resources, such as Amazon Web Services and OpenStack, for storing and analyzing large data from social media. They propose a solution based on a multi-node OpenStack cloud system that enables Hadoop access and use within the cloud environment. Siagian et al. [31] concentrate on improving the architecture of a video surveillance database system utilizing OpenStack Swift cloud storage in an OpenStack cluster. By dynamically adjusting storage allocation for data acquisition from monitoring devices, they assess the scalability and dependability of the system for large-scale video surveillance. Kulkarni et al. [50] propose the implementation of EC in OpenStack Swift to improve storage efficiency, reliability, and fault tolerance in cloud storage systems, highlighting the need for adaptable storage methods to meet the growing data storage demands.

2.5. Data Movement

Tanwar et al. [28] propose the Distributed Linear Order Partition (DLOP) method for data partitioning and deployment. To facilitate the administration of dispersed data, the authors present two partitioning strategies and an indexed data query system. The research validates the efficiency of the proposed storage strategy by demonstrating its scalability and reduced hardware needs. Zhou et al. [51] present a data placement scheme that employs top-down transmission and an enhanced metaheuristic algorithm to manage pipelined data transmission to increase the storage network's lifetime and reduce data insertion time. Feng et al. [32] present MDTUpdate, a double tree-based approach for effective multi-block updates in heterogeneous clusters, which achieves quicker updates and reduced operational costs than previous methods. Facenda et al. [52] propose streaming codes that are adaptable and designed particularly for a three-node network. These codes address infrequent packet loss and ensure on-time delivery by adapting to loss patterns, employing symbol estimates, and encoding at various rates.

After reading all of these studies, we can say that different storage systems were used to apply data using different EC policies and EC schemes. However, to the very best of our knowledge, no research studies have focused on finding time efficiency in terms of I/O operations of EC schemes using standard datasets.

3. Background

3.1. Overview of OpenStack Swift and Erasure Coding

OpenStack Swift [53] is the component that is in charge of object storage services. It offers a solution that is both scalable and long-lasting for the storage of enormous volumes of unstructured data. It does this by segmenting the data into blocks and spreading those blocks along with parity pieces over various storage nodes. Erasure Coding (EC), such as Reed-Solomon (RS) coding, is one example of this.

EC helps improve data security and efficiency. In the event that there is a data loss, the system is able to recreate the original data by piecing together the surviving pieces. EC protects OpenStack Swift data against hardware failures and other types of data loss. Access limitations and encryption further strengthen OpenStack Swift's already robust security protections.

3.2. Reed-Solomon (RS) for OpenStack Swift

Cloud storage systems like OpenStack Swift employ Reed-Solomon (RS) coding. RS algorithms provide distributed storage fault tolerance and data integrity. The OpenStack Swift system uses RS codes. RS codes secure and retrieve OpenStack Swift cloud data. OpenStack Swift divides files into data chunks and computes parity pieces using RS coding. Coding settings determine data and parity chunks. OpenStack Swift's RS coding method permits file reconstruction even if certain chunks are missing or unavailable. RS codes are ideal for cloud storage because of their fault tolerance. OpenStack Swift uses RS decoding to rebuild the original file after retrieving data and parity chunks from the storage nodes. Mathematical procedures on chunks restore lost or distorted data during decoding. OpenStack Swift RS codes use the storage efficiently. OpenStack Swift may reduce data redundancy overhead and provide fault tolerance using RS coding. Choosing data and parity pieces depending on fault tolerance needs maximizes efficiency.

3.3. SimEDC

A simulator for evaluating Erasure Coding (EC) in cloud storage systems. We can evaluate EC performance with the help of SimEDC [33], a Python simulator developed for this purpose. It provides users with the ability to adjust EC schemes and system parameters, as well as generate synthetic workloads for the purpose of evaluating system performance in a variety of contexts. Configuring the simulator's settings, such as the data center architecture, erasure codes, redundancy placement, and subsystem failure patterns, is necessary. SimEDC can offer dependability measures like the probability of data loss, repair efficiency, and bandwidth reduction by running simulations with various configurations. This enables the investigation and assessment of erasure-coded network preservation in various settings and scenarios.

4. Methodology

In our work, we experiment with different datasets for benchmarking Erasure Coding (EC) schemes. Figure 1 illustrates the workflow of our work. Firstly, we plan to select a suitable EC technique for our later experiments. Secondly, we create an experimental testbed for benchmarking EC schemes which is illustrated in Figure 2. Then we choose datasets which are followed by creating our own dataset and using an existing one. Thirdly, our method follows different implementations for our desired results i.e. analyzing data.

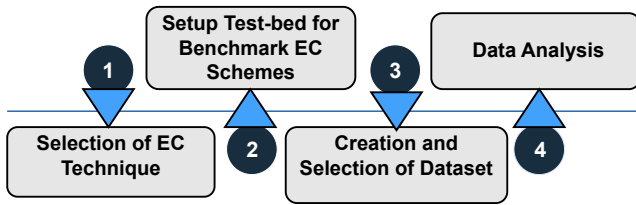


Figure 1: Workflow of our proposed work

4.1. Selection from Existing Erasure Coding Techniques

Our work prioritizes selecting suitable Erasure Coding (EC) techniques for assessing time efficiency and fault tolerance in OpenStack Swift cloud storage systems. EC techniques are employed in data storage and transmission systems to improve reliability and fault tolerance. They offer a means to safeguard data against the loss of specific storage components or network links by producing supplementary redundant information. Several widely utilized erasure EC techniques include XOR-based Erasure Codes [15], Fountain Codes [16], Luby Transform (LT) [17] Codes and Reed-Solomon Codes (RS).

XOR-based erasure codes, also known as XOR-based parity codes, belong to a category of EC techniques that employ bit-wise XOR operations to calculate parity information. These codes find common use in RAID (Redundant Array of Independent Disks) systems, particularly RAID 5 and RAID 6. While XOR-based codes are relatively simple and efficient, they have limitations when it comes to withstanding multiple simultaneous failures. Fountain codes, on the other hand, are a type of erasure code designed to facilitate efficient and reliable data transmission over unreliable or lost networks. They excel in scenarios involving smaller data sets. However, as the data size increases, the efficiency of fountain codes diminishes, and other erasure coding techniques, such as systematic codes like RS, may deliver better performance. LT code, which is a variant of fountain codes, utilizes a random binary generator matrix to generate encoded packets. While LT codes offer a limited error correction capability, they are not as powerful as erasure coding techniques like RS codes. We follow specific criteria to ensure a fair evaluation, considering popularity, extensive use, and compatibility with OpenStack Swift infrastructure. RS codes are chosen as the EC technique due to their vital role in providing fault tolerance and data integrity in OpenStack Swift. By adhering to these selection criteria and leveraging Reed-Solomon codes, we aim to assess EC algorithms relevant to OpenStack Swift's cloud storage system, emphasizing high availability, durability, and reliability.

4.2. Methods Used to Setup the Testbed Environment

Local Testbed EC Setup: We set up the local testbed environment by installing OpenStack Swift on virtual machines (VMs) and configuring Erasure Coding (EC) schemes. Storage policies are defined, OpenStack Swift rings are built,

and EC containers are created for each EC scheme (Reed-Solomon(RS) (5+3), (7+5), and (10+4)). Different VMs represent distinct EC schemes, allowing for data storage and retrieval evaluation.

Remote Testbed EC Setup: We obtain a remote VM to expand the evaluation of the cloud storage system. Then we install OpenStack Swift on the remote VM and configure multiple storage policies to accommodate different Erasure Coding (EC) schemes within a unified environment. Finally, we create OpenStack Swift rings and EC containers for each storage policy to enable comparative analysis and evaluation of the performance of different EC setups.

Simulator (SimEDC) for Testbed: We incorporate a simulator called SimEDC into the testbed to assess the fault tolerance of EC schemes. SimEDC generates synthetic workloads and provides reliability metrics such as the probability of data loss, repair efficiency, number of megabytes of data loss, bandwidth reduction, and single-chunk repair ratio. It offers flexibility to add new EC schemes and modify existing ones, allowing for comprehensive system performance assessment in various settings.

By utilizing these methodologies and incorporating SimEDC, the research aims to evaluate the time efficiency and fault tolerance of EC schemes in OpenStack Swift cloud storage systems.

4.3. Datasets for Testbed

As our analysis demands working with not only different sizes of data but also a large number of files to benchmark the EC schemes thoroughly. For making our needs mitigated we have created our dataset to perform data analysis. The dataset we create, not only has a range of files we need for data analysis, but it also consists of 4 different types of files which include text, image, video, and audio files. Secondly, to mitigate the need to use a vast number of files, we are using a benchmark dataset named COCO-17 [54].

4.4. Approaches for Data Analysis

For our desired result we analyze the dataset with a different approach. For instance, to check the time efficiency of EC we try to upload data on the cloud-based storage system OpenStack Swift. We then download and delete files from OpenStack Swift. For each function, we measure the time to upload, download and delete time. We repeat this method for three EC schemes which are based on Reed-Solomon (RS) (5+3), (7+5), and (10+4). We try these functionalities both for remote and local testbeds. The testbed also includes simulator SimEDC which gives us the result for fault tolerance of EC for various EC schemes.

5. Dataset

A dataset refers to a structured collection of data that is organized and stored for specific purposes such as analysis, research, or information retrieval. It can be composed of various types of data, including text, numerical values, images, audio, or video. We use two different datasets to work on our

problem. One is our own created dataset named MCSD-100 [55] and another one named COCO-17.

5.1. Multitype Cloud Storage Dataset (MCSD-100): A Comprehensive Collection of 100 Text, Audio, Image, and Video Files

We create MCSD-100 [55] to solve the task that we work on in this paper. The aim of creating a dataset for object storage performance testing is to establish a baseline result. In a previous study, Noor et al.[56] used the MSCOCO dataset and semEval2017 dataset for object storage optimization. However, there are no existing datasets to evaluate object storage performance. Most literature shows tests with different file sizes and data types but lacks file references or data used. This inspired the creation of a dataset containing all file sizes and data types for resource allocation and optimization tests. This self-created dataset is carefully curated to evaluate the performance of the OpenStack Swift cloud storage system, specifically regarding its Erasure Coding (EC) policy and fragmentation capabilities. We face challenges in finding an existing dataset that meets our criteria. So we compile a diverse range of data types and sizes, representing real-world scenarios. The dataset includes 25 files each of text, mp3, jpeg, and mp4 formats, with sizes reflecting common file size distributions encountered in cloud storage environments. By analyzing the behavior of these files under different EC schemes, we gain valuable insights into OpenStack Swift's storage mechanisms. The dataset is openly available for reproducibility and further studies. Our findings highlight OpenStack Swift's efficiency in handling different file types and sizes, making it suitable for enterprise data management, multimedia content platforms, image hosting, long-term data preservation, and collaborative content creation.

5.2. COCO-17 (Common Objects in Context-17)

The COCO-17 [54] dataset is widely used in computer vision, providing annotated photos across 17 categories for object identification and picture interpretation tasks. It offers diverse real-world scenarios, accurate annotations and serves as a benchmark for performance comparisons. We utilized a subset of 3000 photos for our investigation, enhancing our analysis while managing resources. COCO-17 improves our understanding of object storage systems and contributes to advancements in the field of data management and storage.

6. Experimental Evaluation

6.1. Experimental Setup

The experimental setup consists of two testbed environment setups, datasets for the testbeds, and testbed simulations implementation. For the testbed environment after installing OpenStack Swift for both the local and remote testbed, we set up EC for both environments. Then for the setup dataset for the testbed, we use some pipeline to upload, download, and delete data on OpenStack Swift and then

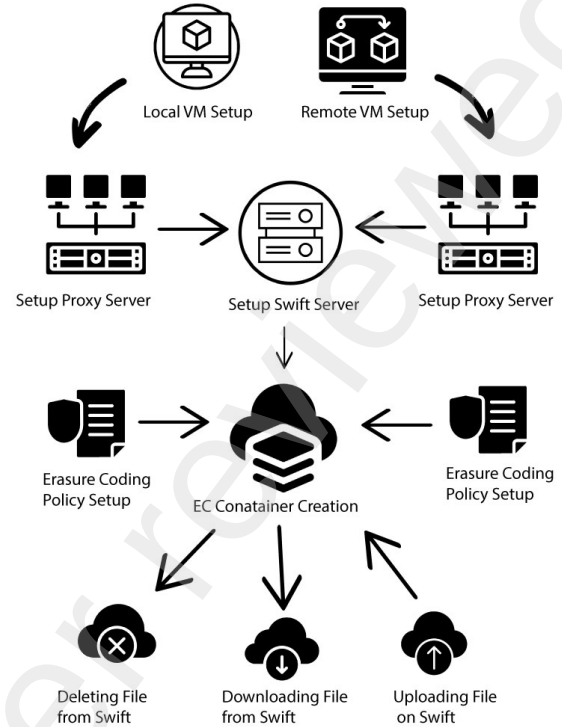


Figure 2: Testbed environment setup

analyze the results from the output. We also used a simulator to measure the fault tolerance of EC.

Testbed Environment Setup: To set up Erasure Coding (EC) in OpenStack Swift, we installed OpenStack Swift locally and created an EC policy [57] description file. The file specifies the EC method, the number of data and parity fragments, and the object segment size. We used the OpenStack Swift ring [58] builder to create a ring for each policy, which defines the cluster layout and ensures the even distribution of fragments. The ring builder also determines the number of copies needed for data availability. After distributing the ring file to storage nodes, we prepare and sent the data to the OpenStack Swift object storage system using the OpenStack Swift API. Additionally, we implement our benchmark data (COCO-17) and our dataset on a remote server using SSH for network access. This secure connection allowed us to configure OpenStack Swift and EC on the remote server. By modifying the `Swift.conf` file, we define three distinct EC policies (10+4, 7+5, and 5+3) with different parameters. We specifically choose 10+4, 7+5, and 5+3 as for swift's replication number. In our SAIO (Swift All In One) we use 3 replicators and also consist of 1 original. As for the 10+4 scheme, we can have 10 data fragments and 4 parity fragments to reconstruct any data, if it gets lost. Swift uses most cases 3-4 replicators for data backup and disaster recovery, where erasure coding is used for reconstructing data from the incident. In these 3-4 replicators persist with at

least one parity node of EC to reconstruct the data. Which is the first thing to choose not more than 5 and less than 3 parity fragments based Scheme. Using the OpenStack Swift API or command-line tools, we can now assign these policies to specific storage containers or objects, providing flexibility based on our data needs. Following, the environment has been set up in two kinds of servers. One in local and one in remote. The local server consists of a 4-core CPU along with 8GB of RAM with HDD for storage, on the other hand, the remote server consists of a 2-core CPU with 4GB of RAM and SSD for storage.

Datasets for Testbed: We analyze the behavior of different data types with various EC regulations for upload, download, and deletion time frames. We set up EC policies in our OpenStack Swift server and created an OpenStack Swift ring to determine storage locations. An AUTH token is generated using the `curl` command for authentication. We establish an EC container to manage data and organize it. We develop Python scripts for analyzing upload, download, and delete times. OpenStack swift API works with the Python script and data pipeline maintained with this. The upload script uses the `curl` command to upload files and measure the time taken. The download script downloads files from OpenStack Swift and calculates the download time. The delete script removes files from OpenStack Swift and measures the time taken. These scripts allow us to analyze data locally and remotely based on time. After fetching the data to the object server, EC stores the data by dividing it into fragments and turning it into hash and stores.

Testbed Simulator Setup: SimEDC is a discrete-event simulator designed for analyzing erasure-coded data center storage reliability. It follows a process involving configuration, initialization, event scheduling, event processing, repair and redundancy techniques, data loss metrics collection, simulation termination, and result analysis. SimEDC allows researchers to customize simulations and evaluate the dependability of erasure-coded storage under various scenarios.

6.2. Experimental Results on Upload, Download, and Deletion

Figure 3 analyzes the upload time of a benchmark dataset (COCO-17) on a remote OpenStack Swift server. The x-axis represents upload time in seconds, and the y-axis shows the size range in KB. Three bars are present: blue for 10+4 data fragmentation, red for 7+5, and yellow for 5+3. Within the 0 KB to 150 KB range, the 10+4 bar exhibits the longest upload time, approximately 0.10 seconds, while the red and yellow bars take around 0.07 and 0.05 seconds, respectively. This relationship holds across other ranges as well. The 10+4 data fragmentation consistently takes the longest time, while 5+3 takes the shortest. The upload time increases with larger data size ranges. Overall, the graph demonstrates the relationship between upload time and data size range, with 10+4 taking the longest time and 5+3 taking the shortest.

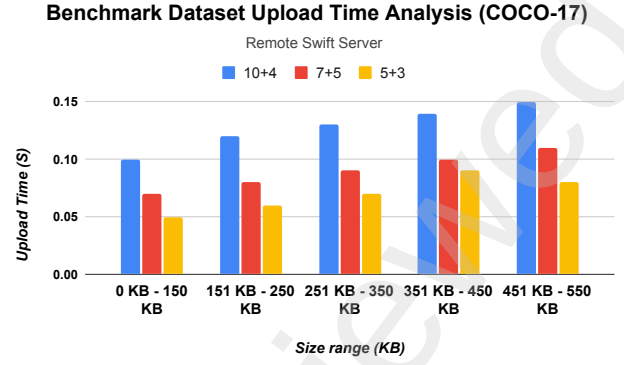


Figure 3: Benchmark dataset (COCO-17 remote OpenStack Swift server)

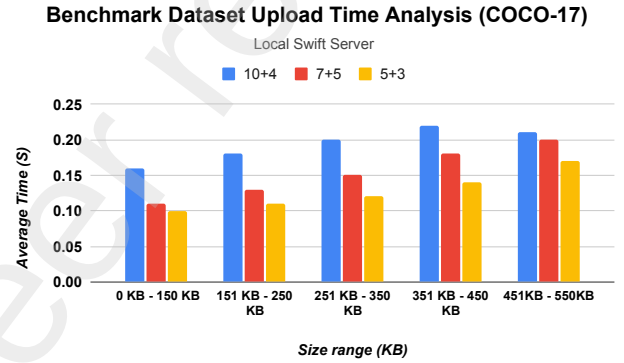


Figure 4: Benchmark dataset (COCO-17 local OpenStack Swift server)

Figure 4 analyzes the upload time of a benchmark dataset (COCO-17) on a local OpenStack Swift server, with the x-axis representing the average time in seconds and the y-axis indicating the size range in KB. The bars, red and yellow for 7+5 and 5+3 data fragmentation respectively, and blue for 10+4, demonstrate the relationship between upload time and size range. Within the range of 0 KB to 150 KB, the 10+4 bar exhibits the longest upload time, while the red and yellow bars show shorter times. This pattern persists across other ranges, except for the range of 451 KB to 550 KB where the time for 10+4 decreases slightly. The 7+5 data fragmentation consistently shows the shortest time, followed by 5+3. Overall, the graph indicates that 10+4 takes the longest time up to the range of 450 KB, with a slight decrease in the 451-550 KB range, while 5+3 exhibits the shortest time. However, it is seen

In Figures 3 and 4, the non-linear increase in performance with file size is primarily attributed to bandwidth limitations, indicating a bottleneck in data transfer. To look out for the bottleneck, we find out that, while we populate the enormous files, the upload time increases. It occurs as a result of the bottleneck caused by the local testbed's low physical hardware performance and restricted bandwidth.

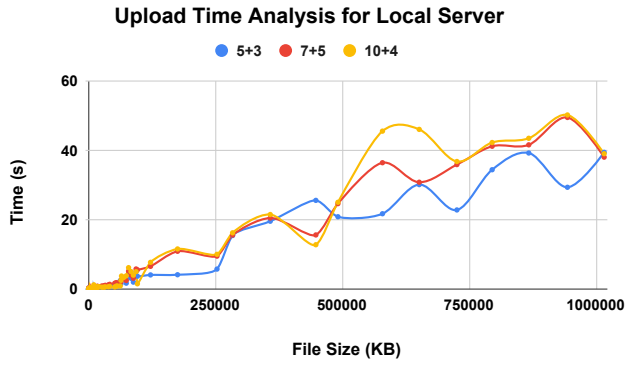


Figure 5: Upload time analysis

Figure 5 compares upload times for our dataset (MCSD-100) on a local server, ranging in size from 100 KB to 10,00,000 KB, for the 5+3, 7+5, and 10+4 EC policies. The x-axis represents the average time in seconds, and the y-axis represents the size range in KB. The graph reveals that upload time increases as the data file size grows for all three policies. Comparing the lines for the three policies, it is evident that higher fragmentation leads to longer upload times which decreases the time efficiency of OpenStack Swift.

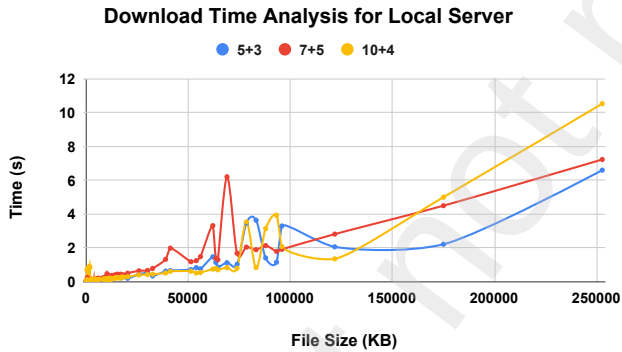


Figure 6: Download time analysis

Figure 6 compares the download times of three erasure coding (EC) policies—5+3, 7+5, and 10+4—across a range of file sizes from 100 KB to 2,50,000 KB. The x-axis represents the average time in seconds, and the y-axis represents the size range in KB. As file size increases, download time increases for all three policies. Comparing the line graphs, it is clear that higher fragmentation levels (10+4) result in longer download times on the OpenStack Swift storage system in comparison to the lower fragment.

Figure 7 compares the delete timings of three erasure coding (EC) policies—5+3, 7+5, and 10+4—across a range of file sizes from 100 KB to 3,50,000 KB on a local server.

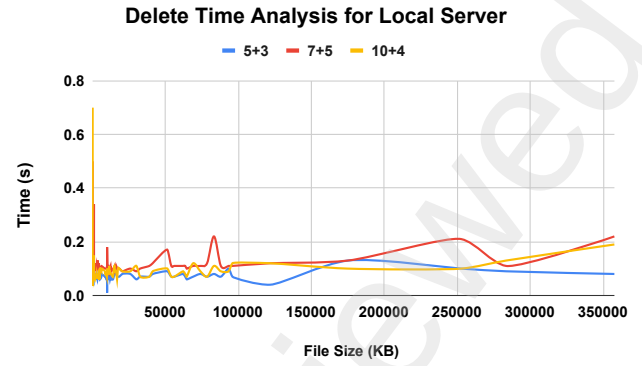


Figure 7: Delete time analysis

The x-axis represents file sizes in KB, and the y-axis represents delete time in seconds. Each data point corresponds to the delete time for a specific file size. Analyzing the graph's data, it observes that the average deletion time for files on OpenStack Swift using these three EC policies is approximately 0.0952 seconds.

Upload Time Comparison Graph for Remote Server

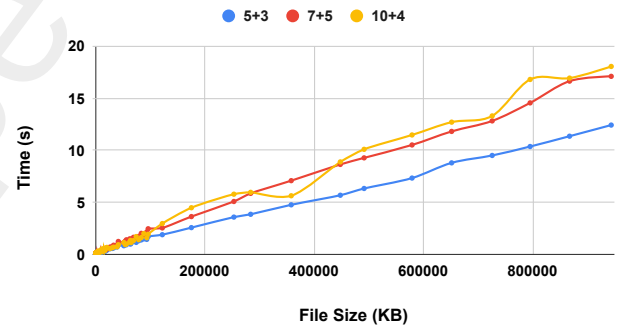


Figure 8: Upload time comparison

Figure 8 compares upload times for our dataset MCSD-100, ranging from 100 KB to 8,00,000 KB, on a remote server using the 5+3, 7+5, and 10+4 EC policies. The x-axis represents file sizes in KB, and the y-axis shows upload time in seconds. Increasing file sizes result in longer upload times for all three policies. Comparing the line graphs reveals that larger data fragmentation leads to extended upload times on the OpenStack Swift storage system.

Figure 9 compares download times for 100 files ranging from 100 KB to 2,50,000 KB on a remote server using the 5+3, 7+5, and 10+4 EC policies. The y-axis represents time in seconds, and the x-axis represents file sizes in KB. Download times increase as file sizes increase for all three policies. Comparing the line graphs reveals that higher levels of fragmentation (10+4) result in longer download times.

Figure 10 compares delete times for 100 files ranging from 100 KB to 1,50,000 KB on a remote server using the 5+3, 7+5, and 10+4 EC policies. The y-axis represents

Download Time Comparison Graph for Remote Server

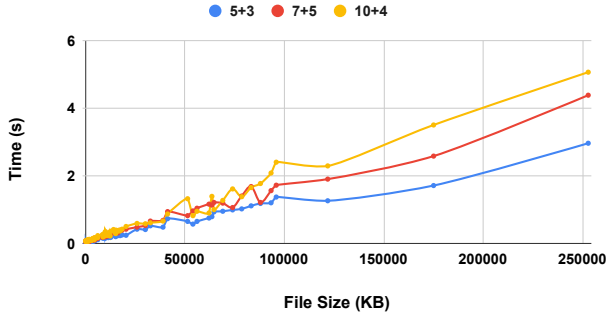


Figure 9: Download time comparison

Delete Time Comparison Graph for Remote Server

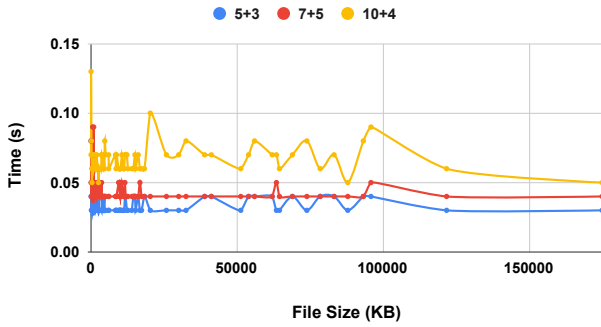


Figure 10: Delete time comparison

delete time in seconds, and the x-axis represents file sizes in KB. Each data point on the graph corresponds to the deletion time of a specific file size. Upon analyzing the data, it was found that the average delete time for the three EC policies on the OpenStack Swift storage system is approximately 0.0481 seconds. If we compare the two downloading time analyses of local and remote servers, it is visible that, the ratio of 5+3, 7+5, and 10+4 schemes to download data differences is similar from one to another. On the other hand, the ratio of these schemes to write data or download data differences has a huge time gap for the bigger scheme (10+5), which is 3.8 seconds. where in remote it is less than 0.75 seconds. This analysis is crucial to find out hardware and I/O impacts in reconstruction. Downloading data is synonymous with writing data to a storage system. We try download time analysis in our system to measure network and storage capacity, and it is clear in Figure 7 and Figure 10 that bandwidth and storage device variances have an impact on data writing as well as data reconstruction. Which is an impactful finding for fault tolerance.

6.3. Data Analysis of Waiting Time for Remote Testbed

Table 2 displays the real-time, user time, system time, and waiting time for each of the 20 data files that we transferred to OpenStack Swift from a distant virtual machine

to study waiting times. According to the results from the table, we can plot the waiting time. Figure 11 demonstrates how waiting times grow exponentially as size increases. Therefore, larger files require greater waiting time.

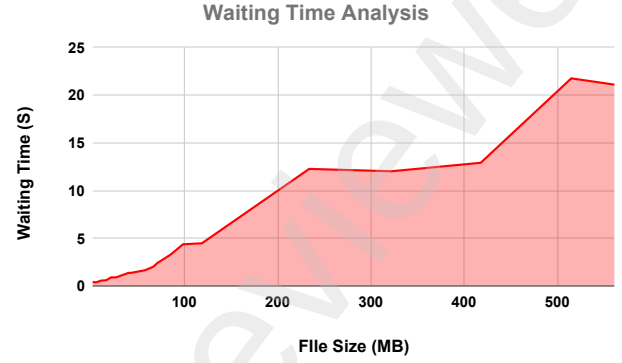


Figure 11: Waiting time analysis

6.4. Results from Testbed Simulator: SimEDC

Table 3 we project output which is generated by executing the Python script `simedc.py` with various arguments. This script is designed to simulate EC along with storage systems by taking into account various parameters such as the quantity of data chunks, parity chunks, and the coding scheme utilized. The absence of patterns with a probability of zero indicates that there were no instances where the loss of data was certain. A reduced PDL value indicates a decreased probability of data loss within the simulated system. A greater value of the RE metric signifies a wider range of potential error and suggests that the estimated mean may lack precision. A reduced Non-Overlapping Maximum-Distance-to-Loss (NOMDL) value denotes an enhanced efficacy in the utilization of storage capacity while encountering data loss. In a system devoid of any obstructed requests, all data and repair requests were effectively attended to, and no instances were observed where the requested data was either unavailable or inaccessible. A reduced repair ratio for a single chunk implies that the system is capable of effectively restoring and retrieving data without necessitating the reconstruction of individual chunks.

6.5. Results from Scenario generated test:

The simulator shows some results based on the imaginary scene which are related to power outage, broken pipelines, rack fails, rack repair, node fail, node transient fail, node transient repair, disk fail, and disk repair mostly. We try to create similar events in our VM to test out the data recovery process to trigger the EC reconstruction. We remove objects from the outside, instead of using the OpenStack CLI or API to create an event. We also delete nodes and remove nodes from the designed proxy and network for logging missing in the OpenStack. After that, we trigger the OpenStack Swift auditor to find out the missing. later on, we manually trigger `reconstruction.py` from the swift run

Data size (MB)	Real time (s)	User time (s)	System time (s)	Waiting time (s)
1.2	0.459	0.009	0.031	0.419
4.4	0.392	0	0.024	0.368
10.5	0.595	0.012	0.021	0.562
15.7	0.64	0.008	0.033	0.599
20.8	0.948	0.004	0.049	0.895
26.5	0.967	0.038	0.028	0.901
30.8	1.155	0.026	0.071	1.058
39.9	1.477	0.016	0.083	1.378
42.2	1.486	0.042	0.077	1.367
57.4	1.781	0.037	0.096	0.648
66.1	2.15	0.029	0.114	2.007
70.6	2.551	0.036	0.115	2.4
85.2	3.496	0.078	0.102	3.316
98.1	4.579	0.051	0.163	4.365
118.4	4.713	0.071	0.166	4.476
233.5	12.845	0.145	0.428	12.272
321.5	12.758	0.171	0.565	12.022
417.5	13.913	0.187	0.817	12.909
514.7	22.926	0.268	0.896	21.762
560.9	22.341	0.31	0.926	21.105

Table 2

Upload time, real-time, user time, system time, and waiting time for different size data

Data Fragment	VARIABLES					
	Num zero	PDL	RE	NOMDL (bytes)	BR	Single- chunk repair ratio
5+3	0	4.719e-06	158.90	1.931e-06	0.00000e+00	0.000
6+4	0	1.959e-06	142.23	2.503e-06	0.00000e+00	0.000
7+5	0	1.131e-06	166.43	1.532e-06	0.00000e+00	0.000
8+7	0	7.203e-10	164.03	1.271e-06	0.00000e+00	0.000
9+6	0	1.068e-26	196.03	1.430e-06	0.00000e+00	0.000
10+4	0	4.643e-84	195.80	2.253e-06	0.00000e+00	0.000
10+7	0	2.506e-12	195.93	1.716e-06	0.00000e+00	0.000
11+8	0	3.955e-07	187.13	1.820e-06	0.00000e+00	0.000
12+9	0	1.222e-08	181.83	1.192e-06	0.00000e+00	0.000

Table 3

Output simulation table of SimEDC

to reconstruct the data and rebalance the ring with remake rings from the swift bin. We run the unit test and functional test to verify the functionality of workable OpenStack Swift without the missing nodes in the server.

7. Experimental Findings

After analyzing the results of our experiments from Table 4, we can conclude that there is a tradeoff between data and parity fragments in terms of time efficiency. As the number of data and parity fragments increases, the time efficiency of the EC schemes decreases. We observe that the waiting time increases with the size of the file. As the file size grows larger, it takes more time to process and handle the file. Furthermore, the outcome summary of the remaining

data sets demonstrates several key benefits - reduced probability of data loss, optimal storage utilization, effective repair mechanisms, and absence of obstructed requests.

7.1. Comparative Analysis

Table 5 and 6 show the work on different distributed storage systems along with our work. It also highlights how our work is different than any other paperwork. Yunnan et al. [12] worked on Improving Security and Storage Efficiency on HDFS. This paper presents a novel approach utilizing HDFS, network coding, and multi-node reading to enhance the security and storage efficiency of the distributed file system. Through experimentation, the approach demonstrates significant enhancements in file transfer and storage safety. Additionally, compared to the original HDFS model, the proposed method achieves a threefold increase

LOCAL SERVER						
EC Schemes	Upload Time (s)	Percentage of Increased Time for Upload	Download Time (s)	Percentage of Increased Time for Download	Delete Time (s)	Percentage of Increased Time for Delete
5+3	3.832	-	0.713	-	0.09	-
7+5	4.671	21.89%	0.924	29.60%	0.09	0%
10+4	4.921	28.41%	0.944	32.38%	0.09	0%
REMOTE SERVER						
5+3	1.259	-	0.998	-	0.05	-
7+5	1.767	40.32%	1.362	36.47%	0.05	0%
10+4	1.823	44.76%	1.984	98.80%	0.05	0%

Table 4

Server performance analysis using average time for upload, download and delete

Name	Worked on	Object Storage System	Testbed Environment	Techniques	Simulator	Benchmark Testing	File Analysis
Xiaolu et al., 2022 [11]	Failure repair in erasure-coded storage	HDFS	X	Erasure Coding (RS, Azure-LRC)	X	X	X
Ra et al., 2018 [10]	Capacity planning and cost analyzing	Ceph (OpenStack)	VM	X	X	FIO	X
Yehia et al., 2018 [13]	Fault Tolerance	HDFS and Ceph	X	Erasure Coding, Replication	X	TestDFSIO, Rados Bench	X
Akintoye et al., 2018 [59]	Performance evaluation of lightweight cloud storage systems	OpenStack Swift	X	Replication, RS, and FMSR	X	X	Files: 50 MB, 102 MB
Yannan et al., 2015 [12]	Security and storage efficiency	HDFS	Four servers under the public network	Network Coding	X	X	0 - 16 GB
Robert et al., 2012 [14]	Data availability and durability	HDFS	LDAP and Kerberos servers	X	X	X	X
Our work	Time efficiency and fault tolerance	OpenStack Swift	Remote server and local server	Erasure Coding (Reed-Solomon Code)	SimEDC	COCO-17, Own dataset (MCSD-100) and scenario generated test	100 KB - 1 GB

Table 5

Comparative analysis of related works for content comparison and used modules comparison

in file reading speed. Xiaolu et al. [11] introduce FastPR, a proactive repair solution that efficiently combines two repair techniques, migration, and reconstruction, to parallelize the repair process across the storage cluster. FastPR optimally coordinates the relocation of STF node chunks and the decoding of chunks using Erasure Coding. This integration enables a fast and parallelized repair operation. Robert et al. [14] analyze data availability and durability with the Hadoop Distributed File System. He worked on a few parameters like GC (Garbage Collection), Execute Absence, Bugs, Space/object Exhausted, User Application, others(unknown), Block loss, and Probability of losing a

block to check the data availability and durability of HDFS. Comes up with the conclusion that HDFS is (almost) always available and (almost) never loses data.

Ra et al. [10] seek to address inquiries by conducting thorough microbenchmarks, aiming to gain a deeper understanding of Ceph's performance in a hyper-converged cloud environment utilizing all-flash storage. Yehia et al. [13] evaluate the fault tolerance performance of Hadoop Distributed File System (HDFS) [37] and Ceph, focusing on the use of Erasure Coding (EC). It introduces a new EC policy called Dynamic Erasure Coding Policy Allocation (DECPA) to minimize stripes and storage usage. The authors explain how

Name	Time Efficiency	Cost Analyzing	Storage Efficiency	Failure Repair	Data Availability and durability	Fault Tolerance
Xiaolu et al., 2022 [11]	X	X	X	Significantly reduces the repair time for packet size by 37.7-52.3% and 1.9-24.7%	X	X
Ra et al., 2018 [10]	X	Write operations are more expensive than read operations, especially for smaller block size and solution shows significant cost reduction	61%72% of total CPU consumption	X	X	X
Yehia et al., 2018 [13]	X	X	EC is more space-efficient (storage requirement for EC are only 66% of that of 3X replication)	X	X	Read performance in replication is better since EC consumes more time in decoding
Akintoye et al., 2018 [59]	X	X	X	FMSR codes have less response time in repairing a single cloud failure compares to RS and Replication	X	X
Yannan et al., 2015 [12]	X	X	File storage time was extended by about 25%	X	X	X
Robert et al., 2012 [14]	X	X	X	X	HDFS is (almost) always available and (almost) never loses data.	Probability of losing a block on a large cluster is 2.1×10^{-4} in 365 days
Our work	Upload and download time increases with increasing fragments (44.78% and 98.60% respectively)	X	X	X	X	EC is fault tolerant based on simulation

Table 6

Result analysis of related work for different benchmark parameters such as time efficiency, cost analyzing, storage efficiency, failure repair, data availability and fault tolerance

EC works in both systems, including data chunking and reconstruction after node failures. Tests using TestDFSIO and Rados Bench benchmarks are conducted to measure read and write performance and evaluate fault tolerance mechanisms. Results show that HDFS replication outperforms EC in read performance but requires more storage, while Ceph shows similar trends with EC performing better in write operations and requiring less storage.

Akintoye et al. [59] conduct a thorough performance evaluation of lightweight cloud storage systems, comparing Replication, Reed-Solomon (RS), and Functional-Minimum

Storage Regenerating (FMSR) codes. Using a dataset of randomly generated files, they measured response times for file upload, download, and repair operations. The paper presents numerical comparisons of these times, revealing that RS had an average upload time of 33.17 seconds for a 50 MB file, while FMSR took 35.02 seconds and Replication took 65.26 seconds. The authors discuss the impact of emulation on the results, noting that although emulation yields shorter response times, it can still be useful for predicting real-world experimentation outcomes. Additionally, the paper includes

a review and survey of multiple cloud storage systems, providing context for the performance evaluation.

Our paper addresses the challenges of time efficiency and fault tolerance in cloud-based object storage systems, specifically focusing on EC. We analyze existing mechanisms and propose innovative strategies to improve time efficiency through caching, data placement, and scheduling optimizations. We also explore techniques to enhance fault tolerance and ensure data availability. The goal is to minimize access latency and improve overall system performance. Our paper contributes to the field by proposing new approaches for enhanced performance in cloud-based object storage systems.

7.2. Limitation

Our research work encounters several limitations that affect the scope and outcomes of our study. The limitation is the inability to integrate our test bed environment with the simulator, which is intended to demonstrate the fault tolerance capabilities of Erasure Coding. Due to technical constraints, we cannot effectively apply our test bed environment to the simulator, limiting our ability to conduct comprehensive experiments and gather data on Erasure Coding performance in a simulated setting. Additionally, we face a limitation in finding appropriate solutions to improve time efficiency within our proposed system. Despite an extensive literature review, there is a scarcity of research papers addressing this specific aspect. This limitation hinders our ability to implement novel approaches for reducing time inefficiencies. However, these limitations also provide valuable insights for future work. Areas for future exploration include investigating storage scalability for handling larger files, developing a more robust test bed environment for simulating Erasure Coding performance, and conducting further research on innovative approaches to enhance time efficiency.

8. Conclusion and Future Work

Our analysis reveals that as the data fragment size increases, the processing time in the cloud storage system also increases, leading to delays in our system. Our primary goal is to enhance time efficiency by minimizing the processing time for files. Additionally, our examination of fault tolerance data uncovered relative errors in every implemented scheme which can improve by implementing load balancing, scaling, effective resource allocation, and algorithmic optimizations. These measures aim to reduce fault tolerance and enhance the overall reliability and performance of our system. Also, we make our dataset, MCSD-100 public which promotes accessibility and enables future research and collaboration in the field of cloud storage and data management.

OpenStack Swift's object storage system's speed and user experience can suffer from prolonged waiting times in several ways. In system design and optimization, waiting times must be kept to a minimum. Reducing waiting times can boost system responsiveness, resource efficiency, and total system throughput. Through efficient resource management, load balancing, improved scheduling algorithms, and

infrastructure scalability, efforts should be made to reduce waiting times in OpenStack Swift. By putting these ideas into practice, waiting times in OpenStack Swift may be decreased, improving system speed, lowering latency, better resource use, and improving user experience.

Organizing and storing data efficiently in a distributed storage system has been a challenging task. Through EC, it involves dividing the data into multiple parts using parity bits on partitioned disks. To further enhance the data structure, a tree structure data model can be employed, which offers a hierarchical and non-linear organization. This approach improves data insertion practicality and efficiency in the disks and allows for flexibility in node sizes.

References

- [1] J. Noor, M. N. H. Shanto, J. J. Mondal, M. G. Hossain, S. Chellappan, A. A. Al Islam, Orchestrating image retrieval and storage over a cloud system, *IEEE Transactions on Cloud Computing* (2022).
- [2] J. Noor, S. I. Salim, A. A. Al Islam, Strategizing secured image storing and efficient image retrieval through a new cloud framework, *Journal of Network and Computer Applications* 192 (2021) 103167.
- [3] A. Chiniah, A. Mungur, On the adoption of erasure code for cloud storage by major distributed storage systems, *EAI Endorsed Transactions on Cloud Systems* 7 (21) (2022) e1-e1.
- [4] K. M. Greenan, D. D. Long, E. L. Miller, T. J. Schwarz, J. J. Wylie, A spin-up saved is energy earned: Achieving power-efficient, erasure-coded storage., in: *HotDep*, 2008.
- [5] S. Balaji, M. N. Krishnan, M. Vajha, V. Ramkumar, B. Sasidharan, P. V. Kumar, Erasure coding for distributed storage: An overview, *Science China Information Sciences* 61 (2018) 1–45.
- [6] C. Wang, Q. Wang, K. Ren, N. Cao, W. Lou, Toward secure and dependable storage services in cloud computing, *IEEE transactions on Services Computing* 5 (2) (2011) 220–232.
- [7] Y. Dong, Coop-u: a cooperative update scheme for erasure-coded storage systems, *Arabian Journal for Science and Engineering* 43 (12) (2018) 7385–7396.
- [8] H.-Y. Lin, W.-G. Tzeng, A secure decentralized erasure code for distributed networked storage, *IEEE transactions on Parallel and Distributed Systems* 21 (11) (2010) 1586–1594.
- [9] X. Pei, Y. Wang, X. Ma, F. Xu, Efficient in-place update with grouped and pipelined data transmission in erasure-coded storage systems, *Future Generation Computer Systems* 69 (2017) 24–40.
- [10] M.-R. Ra, Understanding the performance of ceph block storage for hyper-converged cloud with all flash storage, *arXiv preprint arXiv:1802.08102* (2018).
- [11] X. Li, K. Cheng, Z. Shen, P. P. Lee, Fast proactive repair in erasure-coded storage: Analysis, design, and implementation, *IEEE Transactions on Parallel and Distributed Systems* 33 (12) (2022) 3400–3414.
- [12] Y. Ma, Y. Zhou, Y. Yu, C. Peng, Z. Wang, S. Du, A novel approach for improving security and storage efficiency on hdfs, *Procedia Computer Science* 52 (2015) 631–635.
- [13] Y. Arafa, A. Barai, M. Zheng, A.-H. A. Badawy, Fault tolerance performance evaluation of large-scale distributed storage systems hdfs and ceph case study, in: *2018 IEEE High Performance extreme Computing Conference (HPEC)*, IEEE, 2018, pp. 1–7.
- [14] R. J. Chansler, Data availability and durability with the hadoop distributed file system, *The USENIX Magazine* 37 (1) (2012) 16–22.
- [15] J. J. Wylie, R. Swaminathan, Determining fault tolerance of xor-based erasure codes efficiently, in: *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07)*, IEEE, 2007, pp. 206–215.
- [16] D. J. MacKay, Fountain codes, *IEEE Proceedings-Communications* 152 (6) (2005) 1062–1068.
- [17] F. Uyeda, H. Xia, A. A. Chien, Evaluation of a high performance erasure code implementation (2004).

- [18] S. Ardalan, K. Raahemifar, F. Yuan, V. Geurkov, Reed-solomon encoder & decoder design, simulation and synthesis, in: CCECE 2003-Canadian Conference on Electrical and Computer Engineering. Toward a Caring and Humane Technology (Cat. No. 03CH37436), Vol. 1, IEEE, 2003, pp. 255–258.
- [19] J. Arnold, Openstack swift: Using, administering, and developing for swift object storage, "O'Reilly Media, Inc.", 2014.
- [20] F. Lombardo, S. Salsano, A. Abdelsalam, D. Bernier, C. Filsfils, Extending kubernetes networking to make use of segment routing over ipv6 (srv6), arXiv preprint arXiv:2301.01178 (2023).
- [21] C. Guo, L. Wang, X. Tang, B. Feng, G. Zhang, Two-party interactive secure deduplication with efficient data ownership management in cloud storage, J. Inf. Secur. Appl. 73 (103426) (2023) 103426.
- [22] S. S. Arslan, Durability and availability of erasure-coded storage systems with concurrent maintenance, arXiv preprint arXiv:2301.09057 (2023).
- [23] G. Levitin, L. Xing, Y. Dai, Optimizing uploading and downloading pace distribution in system with two non-identical storage units, Reliability Engineering & System Safety 231 (2023) 109017.
- [24] D. GEETHA, R. SIVASHANKAR, Time-based proxy-encryption for secure erasure code based cloud storage system.
- [25] Y. Song, Q. Zhang, B. Wang, Fachs: Adaptive hybrid storage strategy based on file access characteristics, IEEE Access 11 (2023) 16855–16862.
- [26] C. Yin, Z. Xu, W. Li, T. Li, S. Yuan, Y. Liu, Erasure codes for cold data in distributed storage systems, Applied Sciences 13 (4) (2023) 2170.
- [27] Q. Wang, X. Ye, X. Luo, L. Li, H. Chen, A distributed data storage strategy based on lops, Arabian Journal for Science and Engineering 47 (8) (2022) 9767–9779.
- [28] J. Tanwar, T. Kumar, A. A. Mohamed, P. Sharma, S. Lalar, I. Keshta, V. Garg, Project management for cloud compute and storage deployment: B2b model, Processes 11 (1) (2022) 7.
- [29] X. Zhao, B. Zhu, Z. Zeng, W. Wang, K. W. Shum, G-update: A group-based update scheme for heterogenous erasure-coded storage systems, in: 2022 IEEE 24th Int Conf on High Performance Computing & Communications; 8th Int Conf on Data Science & Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys), IEEE, 2022, pp. 1135–1140.
- [30] A. B. Nandyal, M. Rafi, M. Siddappa, B. B. Sathish, Improving data services of mobile cloud storage with support for large data objects using openstack swift, International Journal of Advanced Computer Science and Applications 12 (6) (2021).
- [31] P. Siagian, E. Fernando, The design and implementation of a dashboard web-based video surveillance in openstack swift, Procedia Computer Science 179 (2021) 448–457.
- [32] J. Opara-Martins, R. Sahandi, F. Tian, Critical review of vendor lock-in and its impact on adoption of cloud computing, in: International Conference on Information Society (i-Society 2014), IEEE, 2014, pp. 92–97.
- [33] M. Zhang, S. Han, P. P. Lee, Simedc: A simulator for the reliability analysis of erasure-coded data centers, IEEE Transactions on Parallel and Distributed Systems 30 (12) (2019) 2836–2848.
- [34] H. Heo, C. Ahn, D.-H. Kim, Parity data de-duplication in all flash array-based openstack cloud block storage, IEICE TRANSACTIONS on Information and Systems 99 (5) (2016) 1384–1387.
- [35] Y. Rivera, I. Gutierrez, J. Marquez, Fulcrum rateless multicast distributed coding design, IEEE Access (2023).
- [36] D.-J. Shin, J.-J. Kim, Cache-based matrix technology for efficient write and recovery in erasure coding distributed file systems, Symmetry 15 (4) (2023) 872.
- [37] J. Darrous, S. Ibrahim, Understanding the performance of erasure codes in hadoop distributed file system, in: Proceedings of the Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems, 2022, pp. 24–32.
- [38] P. M. Chen, E. K. Lee, G. A. Gibson, R. H. Katz, D. A. Patterson, Raid: High-performance, reliable secondary storage, ACM Computing Surveys (CSUR) 26 (2) (1994) 145–185.
- [39] R. Nachiappan, R. N. Calheiros, K. Matawie, B. Javadi, Enhancing efficiency of proactive recovery in erasure-coded cloud storage systems, Available at SSRN 4123756.
- [40] V. Chouhan, S. K. Peddoju, Investigation of optimal data encoding parameters based on user preference for cloud storage, IEEE Access 8 (2020) 75105–75118.
- [41] M. Abebe, K. Daudjee, B. Glasbergen, Y. Tian, Ec-store: Bridging the gap between storage and latency in distributed erasure coded systems, in: 2018 IEEE 38th international conference on distributed computing systems (ICDCS), IEEE, 2018, pp. 255–266.
- [42] Z. Cheng, L. Tang, Q. Huang, P. P. Lee, Enabling low-redundancy proactive fault tolerance for stream machine learning via erasure coding: Design and evaluation, Available at SSRN 4192493.
- [43] R. Nachiappan, R. N. Calheiros, K. M. Matawie, B. Javadi, Optimized proactive recovery in erasure-coded cloud storage systems, IEEE Access (2023).
- [44] M. Qin, Z. Fu, A heterogeneous hybrid storage method based on ceph erasure code, in: 2021 Asia-Pacific Conference on Communications Technology and Computer Science (ACCTCS), IEEE, 2021, pp. 182–186.
- [45] D. Manini, M. Gribaudo, M. Iacono, Modeling replication and erasure coding in large scale distributed storage systems based on ceph, in: Digitally Supported Innovation: A Multi-Disciplinary View on Enterprise, Public Sector and User Innovation, Springer, 2016, pp. 273–284.
- [46] I. Iliadis, Reliability evaluation of erasure-coded storage systems with latent errors, ACM Transactions on Storage 19 (1) (2023) 1–47.
- [47] L. Pu, J. Shi, X. Yuan, X. Chen, L. Jiao, T. Zhang, J. Xu, Ems: Erasure-coded multi-source streaming for uhd videos within cloud native 5g networks, IEEE Transactions on Mobile Computing (2023).
- [48] M. Emu, A. Haque, M. Mahmood, M. M. Asif, A. R. Tanvir, R. S. Joyeeta, Designing a new scalable load test system for distributed environment, Ph.D. thesis, Brac University (2022).
- [49] S. Kengond, D. Narayan, M. M. Mulla, Hadoop as a service in openstack, in: Emerging Research in Electronics, Computer Science and Technology: Proceedings of International Conference, ICERECT 2018, Springer, 2019, pp. 223–233.
- [50] B. Kulkarni, V. Bhosale, Efficient storage utilization using erasure codes in openstack cloud, in: 2016 International Conference on Inventive Computation Technologies (ICICT), Vol. 3, IEEE, 2016, pp. 1–5.
- [51] A. Zhou, B. Yi, L. Luo, Tree-structured data placement scheme with cluster-aided top-down transmission in erasure-coded distributed storage systems, Computer Networks 204 (2022) 108714.
- [52] G. K. Facenda, M. N. Krishnan, E. Domanovitz, S. L. Fong, A. Khisti, W.-T. Tan, J. Apostolopoulos, Adaptive relaying for streaming erasure codes in a three node relay network, IEEE Transactions on Information Theory (2023).
- [53] O. Sefraoui, M. Aissaoui, M. Eleuldi, et al., Openstack: toward an open-source solution for cloud computing, International Journal of Computer Applications 55 (3) (2012) 38–42.
- [54] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, P. Dollár, Microsoft COCO: Common objects in context (2014). arXiv:1405.0312.
- [55] Mcsd-100 - google drive, <https://drive.google.com/drive/folders/1V9wcqiq9hHwfv9FFRInWS-v7csBLWNkb> (2023).
- [56] J. Noor, R. H. Ratul, M. R. A. Uday, J. J. Mondal, M. S. I. Sakif, A. B. M. A. A. Islam, Sherlock in oss: A novel approach of content-based searching in object storage system (2023). arXiv:2303.02105.
- [57] C. L. A. OpenInfra, Storage policies — swift 2.32.0.dev119 documentation, https://docs.openstack.org/swift/latest/overview_policies.html, accessed: 2023-5-30.
- [58] C. L. A. OpenInfra, The rings — swift 2.32.0.dev112 documentation, https://docs.openstack.org/swift/latest/overview_ring.html#fbclid=IwAR1VEB7woEEnuX1wdoD5QhW5M8-QB0FX3KcaTFd-EvHxivKmxR-Dm4o-s1o, accessed: 2023-5-22.
- [59] S. Akintoye, A. Bagula, Lightweight cloud storage systems: Analysis and performance evaluation.